

Name and Surname: Koketso Mohlala
Student ITS No: 402306615
Qualification: Articulation Diploma to BSC Year of Study: 3rd Semester: 2nd

Assignment due date: 30 September 2026 Date submitted: 24 september 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

- Please tick each box to confirm completion.
- Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
 - Apply Harvard Referencing Style for all citations and references.
 - Assignments should be neatly typed; no handwritten assignments will be accepted.
- For essay-style assignments, please include the following sections:
- Table of Contents
 - Introduction
 - Main Body (with relevant subheadings)
 - Conclusion
 - References
- Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution’s policies on academic integrity and plagiarism.

Student Signature: 

Date: 26 September 2026

Table of Contents

1. Introduction 3

2. Application Overview 4

3. Key Features	4
3.1 My Pantry.....	4
3.2 Add / Edit Ingredient.....	4
3.3 Suggested Recipes (Strict Matching).....	4
3.4 All Recipes & Recipe Detail	4
3.5 Settings	4
3.6 Navigation & UI.....	5
4. Technical Implementation.....	5
4.1 Architecture	5
4.2 Database Design.....	5
4.3 Strict-Matching Algorithm.....	5
4.4 User Interface Components.....	5
5. Important Code Snippets.....	6
5.1 Strict Matching Logic (DatabaseHelper).....	6
5.2 Database Creation & Seeding.....	6
5.3 RecyclerView Adapter	6
6. Screenshots	7
7. Challenges Faced and Solutions.....	14
7.1 App Crashing on Startup	14
7.2 Missing Database Tables	14
7.3 Expiry Date Input Experience	14
7.4 Strict Matching Accuracy	14
8. Database Justification.....	15
9. Conclusion.....	15
10. Submission Details.....	16

1. Introduction

This report documents the development of Smart Pantry Manager, a Java-based Android application created for the Mobile App Development 700 practical assignment. The application helps users manage household ingredients and suggests recipes that can be prepared using only the items currently available in the pantry. The primary goal is to reduce food waste by ensuring that only fully achievable recipes are recommended.

The project was developed using Android Studio with Java, SQLite for local data persistence, and Material Design components for a clean and modern user interface. The colour scheme follows a blue, red and white theme.

2. Application Overview

Smart Pantry Manager is a single-user, offline Android application. Users can maintain a digital pantry of ingredients (with quantity, unit and optional expiry date), browse a library of 18 pre-loaded recipes, and receive strict-matching recipe suggestions based on what they currently have.

Core functionality includes:

- Full CRUD operations on pantry items (Create, Read, Update, Delete)
- Strict recipe matching – only recipes with every required ingredient present in sufficient quantity are shown
- Recipe detail screens showing ingredients and preparation method
- Settings for expiry alerts and preferred units
- Persistent local storage using SQLite

3. Key Features

3.1 My Pantry

The main screen displays all ingredients currently stored by the user in a RecyclerView. Each item shows the name, quantity, unit and optional expiry date. Users can add new items via a floating action button, edit existing items, or delete them. Empty-state messaging is provided when the pantry contains no items.

3.2 Add / Edit Ingredient

A dedicated screen allows users to enter or update an ingredient's name, quantity, unit and optional expiry date. A calendar date picker is used for the expiry field. Input validation ensures required fields are completed before saving.

3.3 Suggested Recipes (Strict Matching)

This is the core feature of the application. When the user opens the Suggested tab, the app evaluates every recipe against the current pantry contents. A recipe is only displayed if the user possesses every required ingredient in at least the required quantity. Partial matches are never shown. Ingredient names are normalised (lower-cased with basic plural handling) so that variations such as “tomato” and “tomatoes” still match correctly.

3.4 All Recipes & Recipe Detail

A complete list of the 18 seeded recipes is available. Tapping any recipe opens a detail screen that displays the full ingredient list and the preparation method.

3.5 Settings

The Settings screen provides a toggle for expiry alerts and a preference for measurement units.

3.6 Navigation & UI

Bottom navigation allows quick switching between Pantry, Suggested, Recipes and Settings. The visual design uses a blue primary colour (#1565C0), red accent (#D32F2F) and clean white/off-white backgrounds with Material Design cards.

4. Technical Implementation

4.1 Architecture

The application follows a straightforward activity-based architecture. MainActivity hosts the bottom navigation and fragment/view switching. Separate activities handle adding/editing ingredients and viewing recipe details. Data access is centralised in a DatabaseHelper class that extends SQLiteOpenHelper.

4.2 Database Design

Three main tables are used:

- **pantry_items** – stores user ingredients (id, name, quantity, unit, expiry_date)
- **recipes** – stores recipe titles and methods
- **recipe_ingredients** – links recipes to their required ingredients and quantities

On first launch the database is created and 18 recipes are automatically seeded. The database version is managed so that schema changes can be applied safely.

4.3 Strict-Matching Algorithm

The method getSuggestedRecipes() in DatabaseHelper builds a map of the current pantry (normalised names → available quantities). It then iterates through every recipe. For each recipe it checks whether every required ingredient exists in the pantry map with a quantity greater than or equal to the amount needed. Only recipes that pass this complete check are returned to the UI.

4.4 User Interface Components

RecyclerView with a custom PantryAdapter is used for the ingredient list. Intents are used to pass item or recipe IDs between activities. Material Components provide consistent styling for buttons, cards, text fields and the bottom navigation bar.

5. Important Code Snippets

The following sections highlight the most critical parts of the implementation.

5.1 Strict Matching Logic (DatabaseHelper)

Location: DatabaseHelper.java → getSuggestedRecipes()

```
// Simplified illustration of the core logic // 1. Build map of pantry (normalised name → quantity) // 2. For each recipe: //    - Check every required ingredient //    - Only include the recipe if ALL ingredients //    are present with sufficient quantity // 3. Return the filtered list of fully achievable recipes
```

This method is the heart of the application's value proposition. It guarantees that a user will never be shown a recipe they cannot complete with the ingredients they currently own.

5.2 Database Creation & Seeding

Location: DatabaseHelper.java → onCreate()

The onCreate method creates the three tables and immediately seeds 18 recipes with their ingredients. This ensures that a fresh install already contains useful content for testing the strict-matching feature.

5.3 RecyclerView Adapter

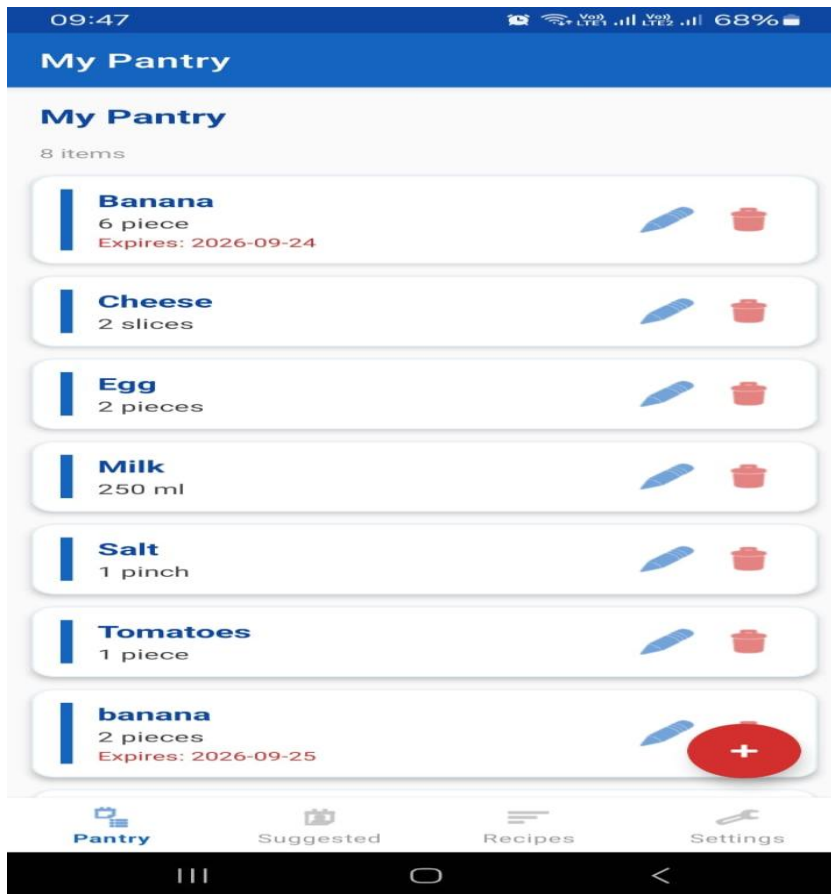
Location: PantryAdapter.java

The adapter receives a list of pantry items from the database and binds each item to a row layout. It also wires the edit and delete button click listeners that trigger the corresponding actions in the parent activity.

6. Screenshots

The following screenshots illustrate the main screens of the application. (Insert actual screenshots from your device or emulator in the final submission.)

1. **My Pantry screen** – list of ingredients with edit/delete actions



2. **Add / Edit Ingredient screen** – form with calendar date picker

09:48 67%

Add Ingredient

Ingredient Name

Quantity

Unit

e.g. pieces, g, ml, cups, tbsp

Expiry Date (optional) 2026-09-24

Tap the calendar icon to pick a date

SAVE

CLEAR EXPIRY DATE

09:48

VoLTE1

VoLTE2

67%

✕

Edit Ingredient

Ingredient Name



Banana

Quantity



6.0

Unit



piece

e.g. pieces, g, ml, cups, tbsp

Expiry Date (optional)



2026-09-24



Tap the calendar icon to pick a date

UPDATE

CLEAR EXPIRY DATE

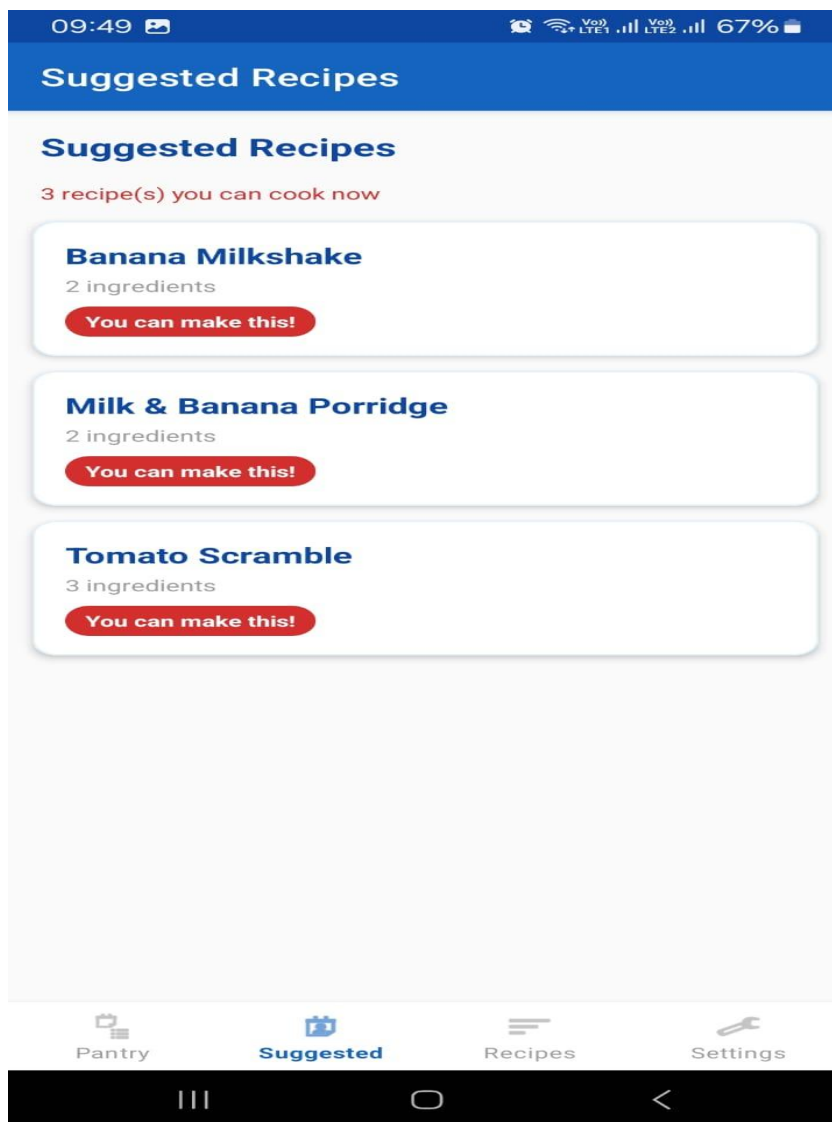
DELETE

III

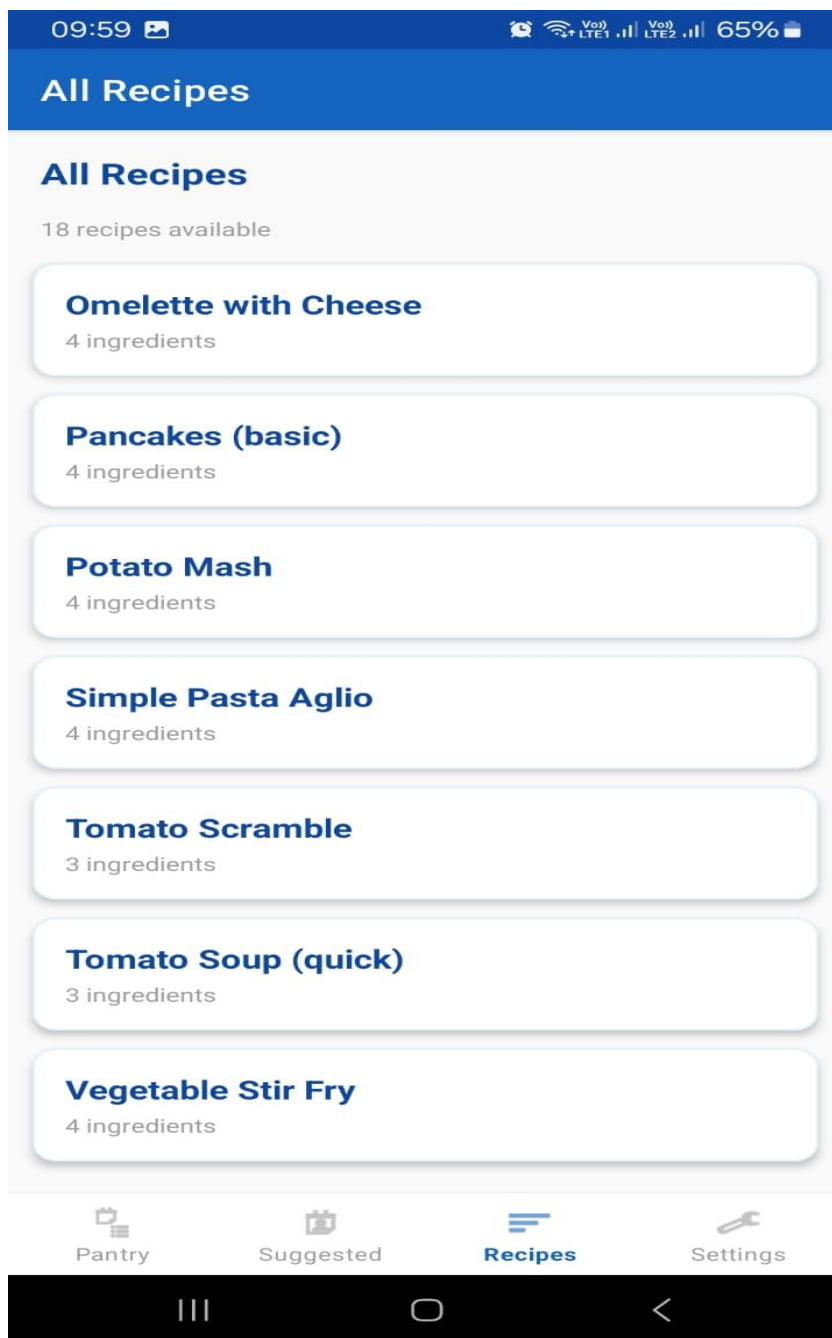
O

<

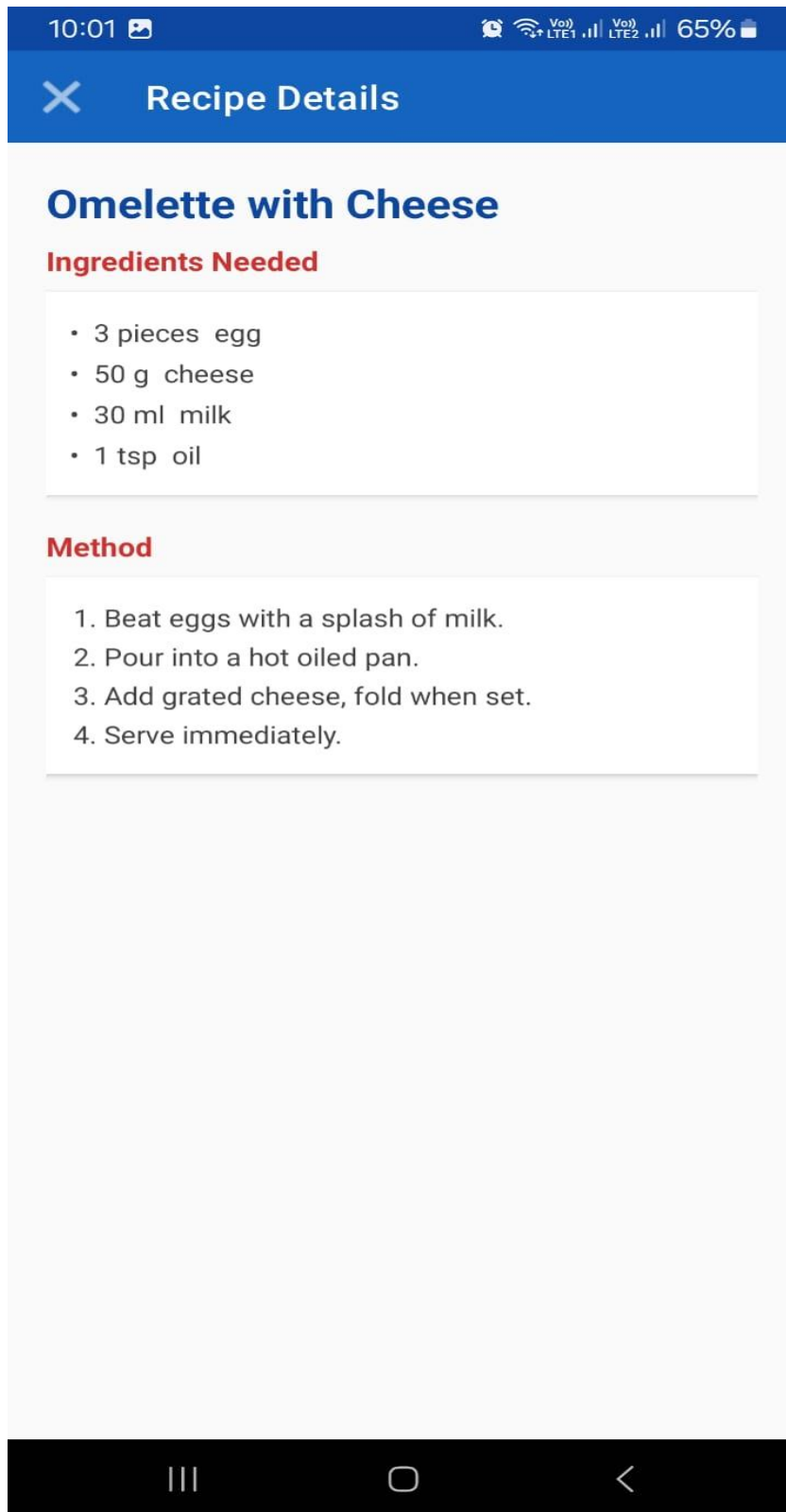
3. Suggested Recipes screen – showing only fully matchable recipes



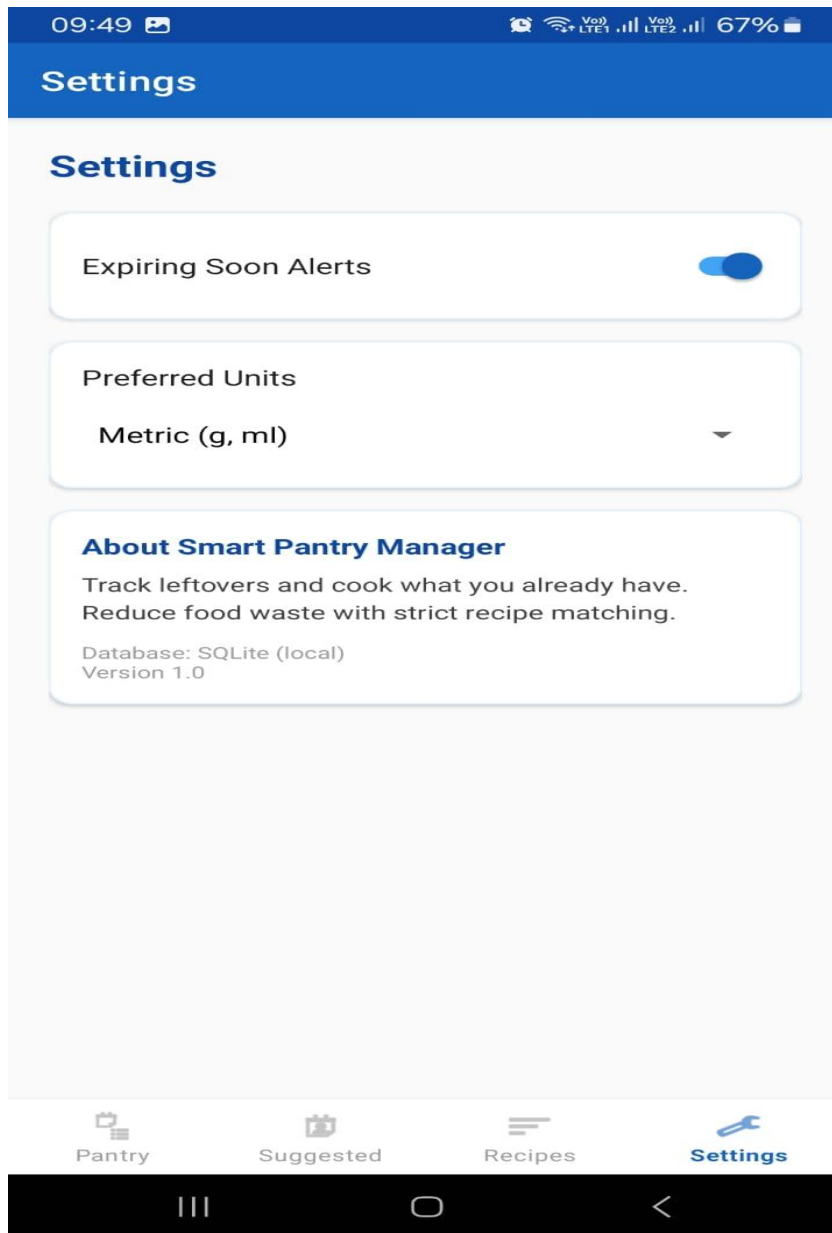
4. **All Recipes list** – complete library of 18 recipes



5. Recipe Detail screen – ingredients and method



6. Settings screen – expiry alerts and unit preference



7. Challenges Faced and Solutions

7.1 App Crashing on Startup

Early versions of the app closed immediately after launch. This was resolved by replacing potentially problematic layout components, improving theme compatibility, adding null-safety checks, and ensuring proper error handling in MainActivity.

7.2 Missing Database Tables

After the first install, some tables were missing, causing “no such table” errors. The database version was incremented and table creation was made more robust with IF NOT EXISTS and automatic recovery logic on startup.

7.3 Expiry Date Input Experience

The original expiry field required manual text entry and looked inconsistent with the rest of the UI. It was replaced with a proper Material calendar date picker, helper text, and a clear-date option.

7.4 Strict Matching Accuracy

Ensuring that ingredient name variations (singular/plural) still matched correctly required careful normalisation logic. Basic lower-casing and plural handling were implemented so that common variations are treated as the same ingredient.

8. Database Justification

SQLite was chosen as the persistence solution for the following reasons:

- **Fully on-device:** The application works completely offline and requires no internet connection or external account.
- **Matches module content:** SQLite is the persistent storage approach covered in the Mobile App Development module.
- **Sufficient for the use case:** A personal pantry and a fixed set of recipes do not require a remote server or complex synchronisation.
- **Reliable persistence:** Data survives application restarts and device reboots, which is essential for a practical pantry tool.

9. Conclusion

Smart Pantry Manager successfully meets the requirements of the Mobile App Development 700 practical assignment. The application provides full CRUD functionality for pantry management, a reliable strict-matching recipe suggestion engine, persistent local storage, and a clean Material Design interface using a blue, red and white colour scheme.

The project demonstrates practical use of Android activities, intents, RecyclerView, custom adapters, SQLite databases, and user input validation. The strict-matching logic ensures that users only see recipes they can actually prepare, directly supporting the goal of reducing food waste.

Future improvements could include barcode scanning for faster ingredient entry, cloud backup, nutritional information, or multi-user support. For the scope of this assignment, however, the current implementation is complete, functional and ready for assessment.

10. Submission Details

GitHub Repository: [koketso35/SmartPantryManager](https://github.com/koketso35/SmartPantryManager)

Video Demonstration: 402306615.mp4